

Implement Client Script & UI Policy (Incident)

Micro Projects

Description : This lab demonstrates how to implement ServiceNow Client Scripts and UI Policies on the Incident form to control field behavior dynamically. In this scenario, Incident fields are evaluated based on configured conditions so that the appropriate fields can become visible, mandatory, or read-only, with Client Script logic used where additional client-side validation or behavior is required.

Timer : 3 hrs

Milestone-1 Creation and Configuration of Incident Form Behavior

Incident Form Preparation:

1. Log in to the ServiceNow instance with admin access.
2. Navigate to Service Desk → Incidents and open an Incident record.
3. Identify the Incident fields whose behavior must change based on an Incident value, such as Category, Subcategory, Priority, or another configured field.
4. Record the expected field behavior for the selected condition before creating the UI Policy and Client Script.

UI Policy Creation:

5. Navigate to All → System UI → UI Policies.
6. Click New to create a UI Policy for the Incident table.
7. Select the Incident table and enter a meaningful short description.
8. Define the condition that determines when the UI behavior should be applied.
9. Configure the UI Policy Actions for the required fields, such as Visible, Mandatory, and Read-only.
10. Save/Submit the UI Policy.

Client Script Creation:

11. Navigate to All → System Definition → Client Scripts.
12. Click New to create a Client Script for the Incident table.
13. Select the appropriate Type, such as onChange or onLoad, based on the required behavior.
14. Select the field that should trigger the script when applicable.
15. Add the required client-side logic to evaluate the Incident value and apply the required form behavior or validation.

Example Client Script Logic:

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == 'critical') {
        g_form.setMandatory('work_notes', true);
        g_form.setVisible('work_notes', true);
    } else {
        g_form.setMandatory('work_notes', false);
    }
}
```

16. Click Submit to save the Client Script.

Milestone-2 Verification of Incident Form Behavior

17. Open an Incident record and test the configured condition.
18. Enter a matching value and verify that the UI Policy changes the configured field state as expected.
19. Change the value to a non-matching condition and verify the alternate field state.
20. Test empty, missing, invalid, or unexpected values where applicable.
21. Verify that the Client Script executes only for the configured form event and field.
22. Save or submit the Incident and verify that the required information is collected correctly.

Verify :

- Matching Incident condition produces the configured UI Policy behavior.
- Non-matching Incident condition produces the alternate configured behavior.
- Mandatory fields are enforced when the configured condition requires them.
- Visible/read-only states change as configured.
- Client Script validation or dynamic behavior executes correctly.
- Empty or unexpected values do not produce unintended form behavior.

Milestone-3 Testing and Maintenance

23. Test the Incident form with representative records covering the main configured conditions.
24. Verify that UI Policy and Client Script behavior work together without conflicting field states.
25. Review the browser/client-side behavior and confirm that the intended form changes occur.
26. Update the UI Policy or Client Script when Incident requirements change.
27. Document the selected field, condition, expected behavior, test result, and any maintenance notes.

Verification Summary:

- Administrator can configure the Incident UI Policy condition and actions.
- Administrator can configure Client Script logic for Incident form events.
- Users see consistent field behavior based on Incident values.
- Required fields are clearly enforced when applicable.
- Dynamic visibility and read-only behavior is applied as configured.
- Changes can be maintained when business requirements change.

Outcome :

By completing this lab, learners gain practical understanding of how ServiceNow Client Scripts and UI Policies can be used together to control Incident form behavior. The exercise demonstrates condition-based field visibility, mandatory-state changes, read-only behavior, and client-side validation, while also emphasizing testing and maintenance of the configured Incident form logic.